INTERVIEW PREPARATION

Automated Analytics

Second and third interviews: technical, then product.

PREPARED FOR	PREPARED BY	DATE
Automated Analytics interviews	Chris Garlick	29 October 2026

Contents

01	Two conversations, two different tests	2
02	How to answer a conversation-style interview	3
03	Laravel: how you structure an application	4
04	Laravel: performance and reliability	5
05	React	6
06	Angular, and how it compares	7
07	Joining Laravel to a React or Angular front end	8
08	AWS for a Laravel application	9
09	A design question, worked through	11
10	Stories to have ready	12
11	Likely questions, with the gist of an answer	13
12	Questions to ask John and Matthew	14
13	What David and Joseph are looking for	15
14	A shape for working through any problem	16
15	Questions worth asking about any problem	17
16	A worked example	18
17	Scenarios to practise	19
18	Building AI features responsibly	20
19	Questions to ask David and Joseph	21
20	Preparation checklist	21

01

OVERVIEW

Two conversations, two different tests

You're through the first round. The next two interviews look at different things, so prepare for them differently.

	Technical interview	Product interview
Who	John and Matthew, two Senior Engineers	David (Product Manager) and Joseph (Engineer)
Format	Conversation about how you build software	Talking through a realistic problem together
Covers	Backend, frontend and your own experience	How you approach a problem you haven't seen
Stack	Laravel, React / Angular, AWS	Whatever the problem needs
Not included	No live coding, no take-home test	No single right answer, nothing to prepare
They're looking for	*How you think*, not perfect answers	Good questions, clear reasoning, sensible scope

What both interviews have in common

Both say the same thing in different words: **show your thinking**. They want to hear how you weigh options, what you'd ask, and why you'd choose one approach over another. A confident *"it depends on X, and here's how I'd find out"* beats a guess dressed up as certainty.

NOW CONFIRMED	The stack is **Laravel, React or Angular, and AWS**. The first guide covered the whole advert; this one goes deeper on those three.
THE PRODUCT	AI-driven marketing, recruitment and analysis software: campaign automation and business intelligence for clients.

02

TECHNICAL INTERVIEW

How to answer a conversation-style interview

With no code to write, your answers *are* the evidence. Give them a shape.

A good technical answer

1 **Restate or clarify**

"Just to check, do you mean for a single server or at scale?" Shows you don't rush in.

2 **Give the default**

What you'd normally do, in one or two sentences.

3 **Name the trade-off**

What it costs, and when you'd choose something else.

4 **Ground it in experience**

"On Typeset I…" or "Last time I hit this…" One concrete example beats three general ones.

5 **Stop**

Leave room for the follow-up. Senior engineers will dig where they're interested.

Instead of	Try
"I'd use Redis."	"I'd cache it in Redis with a TTL, because the data changes hourly. If it had to be exact, I'd invalidate on write instead."
"I don't know."	"I haven't used that directly. My guess is X, because Y. I'd check by Z."
A ten-minute monologue	Two minutes, then: "Do you want me to go deeper on any part?"
"We did X."	"*I* did X." They're hiring you, not your last team.

It's fine not to know

They've said they want how you think, not perfect answers. If Angular or a particular AWS service is new to you, say so plainly, then reason about it from what you do know. That *is* the test.

03

BACKEND

Laravel: how you structure an application

Expect *“talk us through how you’d build…”* and *“where does this logic live?”*

Topic	What to say
Request lifecycle	public/index.php → kernel → middleware → router → controller → response, back out through middleware.
Service container	Binds interfaces to implementations; constructor injection; swap a real client for a fake in tests.
Thin controllers	Validate (Form Request), authorise (Policy), call an action or service, return a resource.
Actions / services	One class per business operation, e.g. LaunchCampaign. Reusable from controllers, jobs and commands.
Eloquent	Relationships, scopes, casts. Eager-load with with(); turn on preventLazyLoading() in development.
Queues	Anything slow or external goes on a queue: sends, imports, report builds, AI calls. Jobs must be idempotent.
Events	Decouple side effects: CampaignLaunched → notify, log, sync. Queue the slow listeners.
Testing	Feature tests through HTTP with RefreshDatabase; Queue::fake, Http::fake; unit tests for pure logic.

```
// Controller stays thin: validate, authorise, delegate
public function store(
    StoreCampaignRequest $request,
    LaunchCampaign $launch,
): CampaignResource {
    $this->authorize('create', Campaign::class);

    $campaign = $launch(
        $request->user()->account,
        $request->validated(),
    );

    return new CampaignResource($campaign);
}
```

> **Say this**
> *"I keep controllers thin and put each business operation in its own action class. That way the same logic runs from an API call, a queued job or an Artisan command, and it's easy to test on its own."*

04

BACKEND

Laravel: performance and reliability

Analytics and automation mean lots of data and lots of background work. Expect a *"this page is slow"* question.

"A dashboard page is slow. What do you do?"

1 **Measure first**

Telescope, Debugbar or APM: how many queries, how long each, and is it PHP time or database time?

2 **N+1 queries**

Hundreds of near-identical queries: eager-load with with() or with-Count().

3 **Slow queries**

EXPLAIN the worst one; add a composite index matching the WHERE and ORDER BY.

4 **Too much data**

Select only needed columns; paginate; aggregate in SQL, not in PHP collections.

5 **Pre-compute**

Dashboards read daily summary tables built by a scheduled job, not raw events.

6 **Cache**

Cache::remember with a TTL for numbers that don't need to be live; tag by account to invalidate.

Background work	Good practice
Retries	$tries and $backoff; retry transient failures, fail fast on bad data.
Idempotency	updateOrCreate, unique keys, or a processed-message table, so a retry can't double-send.
Big jobs	Split into batches (Bus::batch) or chunk; never load a million rows into memory.
Rate limits	Throttle calls to third-party APIs with RateLimiter or job middleware.
Visibility	Horizon for Redis queues, or CloudWatch for SQS; alert on failed jobs and queue age.

> **Your own example: memory on a small server**
> Typeset's production box has under 1 GB of RAM. Large renders were being killed by the kernel (an empty error plus a non-zero exit meant a signal, and dmesg confirmed it). Swap fixed it short term; the proper fixes are fewer concurrent renders and a real timeout. A good story for *"how do you debug production problems"*.

05

FRONTEND

React

Talk about components, state and data fetching, and the mistakes you know to avoid.

Topic	What to say
Components	Small, focused, props in and callbacks out. Composition over deep prop drilling.
State	Keep it as local as possible; lift it up when shared; context for app-wide things like the user.
Server state	Data from the API isn't app state: TanStack Query handles caching, refetching and loading states.
Effects	useEffect is for syncing with outside systems, not for deriving values. Clean up and cancel.
Derived values	Compute during render; useMemo only when it's measurably expensive.
Rendering	Re-renders are usually fine; fix real problems with memo, stable callbacks, or splitting state.
Lists	Stable keys from IDs, never the array index for lists that change.
Forms	Controlled inputs, or React Hook Form with a schema (Zod) shared with TypeScript types.
TypeScript	Type props and API responses; unknown for untrusted data, then validate.

```tsx
// Server state with TanStack Query: caching,
// loading and error handled for you
function CampaignStats({ id }: { id: number }) {
  const { data, isPending, error } = useQuery({
    queryKey: ['campaign-stats', id],
    queryFn: ({ signal }) =>
      api.get<Stat[]>(`/campaigns/${id}/stats`,
                      { signal }),
    staleTime: 60_000,
  })

  if (isPending) return <Spinner />
  if (error) return <ErrorPanel error={error} />
  if (!data.length) return <Empty />
  return <StatsChart stats={data} />
}
```

06

FRONTEND

Angular, and how it compares

They use React *or* Angular, so some of the codebase is probably Angular. Know the ideas even if you've used it less.

Concept	Angular	React equivalent
Building block	Components with a template, class and styles	Function components and JSX
Shared logic	Services, provided through dependency injection	Custom hooks, context
Async data	RxJS Observables; HttpClient returns them	Promises, TanStack Query
Reactivity	Signals (modern) and change detection	useState and re-rendering
Templates	@if, @for, bindings: [prop], (event), [(ngModel)]	JSX with plain JavaScript
Forms	Reactive forms with validators	React Hook Form, Formik
Structure	Opinionated: routing, HTTP, forms, testing built in	A library: you choose the pieces
Language	TypeScript, always	TypeScript by choice

RXJS IN ONE LINE

A stream of values over time. switchMap cancels the previous request when a new one starts, which is ideal for search boxes.

UNSUBSCRIBING	Use the async pipe or takeUntilDestroyed() so subscriptions don’t leak.
CHANGE DETECTION	OnPush plus signals means Angular only re-checks components whose inputs or signals changed.

```
// Search box: debounce, skip repeats,
// cancel stale requests
results$ = this.query.valueChanges.pipe(
  debounceTime(300),
  distinctUntilChanged(),
  switchMap(q => this.api.searchCandidates(q)),
);
```

Say this

“I’ve worked more with React, but the ideas carry over: components, services instead of hooks, and Observables instead of promises. I’d lean on the team’s conventions and get productive quickly.”

07

FULL STACK

Joining Laravel to a React or Angular front end

The seams between front and back are where real bugs live. Show you’ve thought about them.

Concern	Approach
Authentication	Same domain SPA: Sanctum cookie sessions with CSRF. Mobile or third parties: Sanctum or Passport tokens.
CORS	Allow only your front-end origins; credentials only where cookies are used.
Response shape	API Resources for a stable contract; never return raw models with hidden fields leaking.
Validation errors	Laravel returns 422 with field errors; the front end maps them onto the form.
Types	Share a contract: OpenAPI generated from Laravel, then TypeScript types generated from that.

Concern	Approach
Pagination	Cursor pagination for large or live lists; the front end loads the next page on scroll or click.
Long tasks	Return 202 with a job ID; the front end polls, or listens on a websocket (Reverb, Pusher).
Multi-tenancy	Every query scoped to the client's account, enforced server side, never trusted from the front end.

> **A common follow-up: "How would you build an export of a big report?"**
> Don't build it in the request. Queue a job that streams rows to a CSV in S3, mark the export ready, and notify the user with a short-lived signed URL. The page shows *preparing* and then a download link.

08

INFRASTRUCTURE

AWS for a Laravel application

Know which service does what and how they fit together. You don't need to be an ops engineer.

Need	AWS service	Notes
Run the app	EC2, ECS on Fargate, Elastic Beanstalk, or Lambda via Vapor / Bref	Containers on Fargate: no servers to patch, scales on load
Database	RDS or Aurora (MySQL / Postgres)	Multi-AZ for failover; read replicas for reporting
Cache, sessions, queues	ElastiCache (Redis)	Shared across app instances; never store sessions on disk
Queues	SQS, with a dead-letter queue	Laravel's SQS driver; workers scale on queue depth
Fan-out	SNS or EventBridge	One event, several consumers
Files	S3, served via CloudFront	Uploads and exports; signed URLs for private files
Email	SES	Bounce and complaint handling matters for campaign sending

Need	AWS service	Notes
Secrets	Secrets Manager or Parameter Store	Not in .env files committed anywhere
Access	IAM roles, least privilege	The app's role can only touch its own bucket and queue
Monitoring	CloudWatch logs, metrics and alarms	Alarm on 5xx rate, queue age, database CPU

"Walk us through deploying a Laravel app on AWS"

1. **Build**

   CI runs tests, builds one Docker image, pushes it to ECR.

2. **Release**

   Run migrations (backwards compatible), then roll the new image out to ECS behind a load balancer.

3. **Check**

   Health checks gate traffic; a failed check rolls back to the previous task definition.

4. **Scale**

   Web tasks scale on CPU or requests; queue workers scale on SQS queue depth.

> **Stateless web servers**
> Behind a load balancer any request can hit any server, so sessions, cache and uploads must live in Redis, the database or S3, never on the local disk. A classic follow-up question.

09

THINKING OUT LOUD

A design question, worked through

They may ask you to design something on the spot. Here's the shape to follow, using a likely example.

> **"Clients want a weekly email summarising how their campaigns performed."**
> A good fit for their product: scheduled, per client, data heavy, and possibly AI-written.

1. Clarify

1

Who and how many

How many clients and campaigns? Which metrics? Their timezone or ours?

2

What counts as done

Is a plain summary enough at first, or do they want charts and AI commentary?

2. Sketch the flow

3

Schedule

A scheduled command each hour finds accounts whose local Monday 9am has arrived.

4

Fan out

Dispatch one queued job per account, so one slow client doesn't hold up the rest.

5

Build

Read last week's numbers from the daily summary tables; render the email; send through SES.

6

Record

Store a sent_reports row (account, week) with a unique key, so a retry never sends twice.

3. Talk through what could go wrong

7

Late data

Run after the nightly aggregation finishes, or the numbers will be wrong.

8

Failures

Retries with backoff; failed jobs alerted; a resend button for support.

9

AI commentary

Pass the numbers in, ask for structured output, never let it invent figures; cache the result.

4. Close

10

Smallest version

Plain numbers first, AI summary as a follow-up, measured by open rate.

10

YOUR OWN EXPERIENCE

Stories to have ready

Your own experience is one of the three things they'll cover. Prepare short, specific stories.

They might ask	Story to tell
Something you built that you're proud of	**Typeset**: an HTTP API that turns Markdown or JSON into branded PDFs. You designed the schema, built the renderer and front end, and run it in production.
A hard production problem	Renders failing with an empty error on Typeset. You worked out it was the kernel killing the process for memory, fixed it short term, and identified the real fixes.
Learning something new fast	Typeset is written in Rust, and renders with Typst. Neither was your day job.
How you use AI in your work	Claude Code daily: planning, refactoring, writing tests, reviewing. You check what it produces; it speeds you up rather than replacing judgement.
Your Laravel experience	Have two real apps in mind: the problem, how you structured it, one thing you'd do differently.
A disagreement in code review	A time you changed your mind, or changed someone else's, with evidence rather than opinion.
A trade-off you made	Shipping a simpler version first, and what you deliberately left out.

Typeset in one breath

"It's an API I built that turns Markdown or structured JSON into properly typeset PDFs. It's Rust with Typst for the layout, Postgres and Docker, running on a small DigitalOcean server. I use it for real documents, including the one I prepared for this interview."

Be honest about scale

If they ask about traffic or team size on your projects, give the real numbers. Then talk about what you'd change at a larger scale. That shows the thinking they're after.

11

TECHNICAL INTERVIEW

Likely questions, with the gist of an answer

Question	The gist
How do you structure a Laravel app as it grows?	Thin controllers, action classes, Form Requests, Policies, API Resources; domain folders once it's big.
How do you find and fix N+1 queries?	preventLazyLoading in dev, Debugbar or Telescope to spot them, with() and withCount() to fix.
When do you reach for a queue?	Anything slow, external or retryable. Keep the request fast; make jobs idempotent.
How do you test a Laravel app?	Mostly feature tests over HTTP; fakes for queues, mail and HTTP; unit tests for tricky logic.
useEffect: what goes wrong?	Missing dependencies, deriving state in effects, no cleanup, race conditions on fetches.
How is Angular different from React?	Framework vs library; DI and services; RxJS; templates vs JSX; signals.
How would you handle a search box?	Debounce, cancel stale requests (AbortController or switchMap), show loading and empty states.
How do you secure an API for many clients?	Auth, then authorisation per record (policies), tenant scoping, validation, rate limits.
How would you deploy and scale on AWS?	Docker image, ECS Fargate behind a load balancer, RDS, ElastiCache, SQS workers scaled on depth.

Question	The gist
How do you deploy without downtime?	Backwards-compatible migrations first, rolling deploy, health checks, quick rollback.
What makes code maintainable?	Clear names, small units, tests, consistent conventions, and boring choices where possible.
How do you review someone's PR?	Behaviour and tests first, then design, then style; ask questions rather than issue orders.

12

TECHNICAL INTERVIEW

Questions to ask John and Matthew

Engineers enjoy talking about their codebase. Pick three or four.

- Is the front end React, Angular or both? Is there a plan to settle on one?
- How is the Laravel codebase organised: one app, or several services?
- What runs on AWS today: containers, Vapor, EC2? What would you change if you started again?
- How do you handle the heavy background work: queues, scheduled jobs, data pipelines?
- What does testing and code review look like? How often do you deploy?
- Where does AI sit in the code: calls to model APIs, your own models, or both?
- What's the part of the system you'd most like to rewrite, and why?
- What does a new engineer usually find hardest in their first month?

Closing line for the technical interview

"I enjoyed that. I'm strongest on the Laravel side and comfortable across the stack, and I like the mix of data-heavy backend work and client-facing features you've described."

13

PRODUCT INTERVIEW

What David and Joseph are looking for

A Product Manager and an Engineer, one realistic problem, and a conversation. It's a simulation of working with them.

They're testing	What it looks like
Curiosity	You ask about the user and the problem before suggesting a solution.
Product sense	You think about who it's for, why it matters, and how you'd know it worked.
Pragmatism	You suggest a small first version, not the perfect system.
Technical judgement	You can say what's easy, what's hard, and what's risky to build.
Collaboration	You build on their ideas, take hints, and change your mind when they give new information.

Why Joseph is there

An engineer in a product interview is watching for whether you can turn a fuzzy problem into something buildable, and whether you'd be good to work with. Bring the engineering in, but don't jump straight to the database schema.

The most common mistake

Hearing the problem and immediately designing a solution. **Asking questions is encouraged**, so ask them first. The problem as stated is often not the real problem.

14

PRODUCT INTERVIEW

A shape for working through any problem

Not a script to recite. A checklist in your head so you don't skip a step.

Understand

1

The user

Who has this problem? A client's marketing manager, a recruiter, an internal account manager?

2

The problem

What are they trying to get done? What happens today, and what's painful about it?

3

The evidence

How do we know? Support tickets, churn, sales calls, usage data?

4

Why now

What's changed? A big client asking, a competitor, a new capability like AI?

Decide what good looks like

5

Outcome

What changes for the user and the business if we solve it?

6

Measure

One or two metrics: adoption, time saved, retention, fewer tickets.

Explore options

7

Several ways

Two or three approaches, including a non-software one if it fits.

8

Trade-offs

Value against effort and risk for each.

Choose a first step

9 **Smallest useful version**

What can we ship in a week or two that tests the idea with real users?

10 **Risks**

What could go wrong: data quality, privacy, cost, performance, trust in AI output.

11 **Next**

What we'd learn, and what we'd build next if it works.

> **Say this**
> *"Before I suggest anything, can I ask a few questions about who's running into this and what they do today?"*

15

PRODUCT INTERVIEW

Questions worth asking about any problem

Have these ready. You won't use them all; pick the ones that unlock the problem.

About	Questions
The user	Who exactly? How technical are they? How often do they hit this?
Today	How do they handle it now? A workaround, a spreadsheet, a support ticket?
Size	How many clients does it affect? Is it one loud client or a pattern?
Value	Does it affect revenue, retention or support load? Is anyone paying for it?
Success	If we nailed this, what would we see in three months?
Data	What data do we already have? Is it accurate and timely enough to rely on?
Constraints	Deadline? Team size? Anything we can't change, like a third-party API?
Risk	Personal data involved? Anything clients would be upset to see go wrong?
Scope	What's explicitly *not* part of this?

Listen to the answers

The answers are hints. If David says *"it's mostly our three biggest clients"*, that changes the solution. Say so out loud: *"OK, so this might start as something we do for those three, before building it into the product."*

16

PRODUCT INTERVIEW

A worked example

A problem in their world, talked through the way you'd do it in the room.

"Clients say they don't know which of their campaigns are actually working."

Said by David as the opening of the exercise. Below is one way the conversation could go.

ASK	Which clients: marketing or recruitment? Do they look at the dashboard now? What do they do with the answer, pause campaigns, move budget? What does *working* mean to them: clicks, applications, hires, sales?
SUPPOSE	Recruitment clients, mostly. They see clicks and applications but not which campaigns led to hires. Account managers spend hours building this by hand in spreadsheets.
REFRAME	The real problem isn't *more data*. It's connecting a campaign to the outcome the client cares about, and saving the account managers' time.
OPTIONS	A cost-per-hire report by campaign, if hire data exists. Or pull hires from their ATS through an integration. Or an AI summary that highlights the best and worst campaigns each week.
FIRST STEP	Start with the account managers' spreadsheet: build that report for a handful of clients using data we already have. Measure time saved and whether clients act on it.

RISKS	Hire data may be missing or late; attribution is fuzzy when a candidate saw several campaigns; AI summaries must only state real figures.
NEXT	If clients use it, add the ATS integration to make hire data automatic, then the AI summary on top.

> **Notice the pattern**
> Ask, then restate the problem in your own words, then offer options with trade-offs, then pick a small first step and say how you'd measure it. Engineers in the room like hearing *"using data we already have"*.

17

PRODUCT INTERVIEW

Scenarios to practise

Talk each one through out loud for five minutes. These are the kinds of problem they could bring.

Scenario	Angles worth raising
Clients want AI to write their campaign copy	Brand voice, approval before sending, measuring if AI copy performs better, cost per generation.
Recruiters get too many applicants to review	Ranking or summarising candidates; bias and fairness; the human makes the decision; GDPR.
A big client is threatening to leave	Find out why first; one client's fix vs a product change; don't build a one-off without a plan.
Dashboards are too slow for large clients	Pre-aggregated tables, caching, loading in stages; which numbers really need to be live?
Support gets lots of *"why is my number different?"* tickets	Timezones, late data, definitions; show *last updated* explain metrics in the UI.
Clients want to connect their own CRM	Which CRMs most clients use; start with one; webhooks vs polling; what breaks when their API changes.
Campaign emails land in spam	Domain authentication (SPF, DKIM, DMARC), sending reputation, bounce handling, warm-up.
We want clients to log in more often	Why don't they now? Useful alerts and weekly summaries can beat a nicer dashboard.

Ways to prioritise, if they ask

Method	In a sentence
Impact vs effort	Do the high-impact, low-effort things first; question the high-effort ones.
RICE	Reach × Impact × Confidence ÷ Effort: a score to compare ideas fairly.
MoSCoW	Must, Should, Could, Won't: useful for cutting scope on a deadline.

18

PRODUCT INTERVIEW

Building AI features responsibly

They're an AI company. Expect AI to come up in the problem, and have a view.

Question to ask	Why it matters
Does it need to be AI?	Sometimes a rule, a filter or a good default does the job more cheaply and reliably.
What happens when it's wrong?	A bad subject line is fine; a wrong figure in a client report isn't. Match checks to the risk.
Is a human in the loop?	Suggestions a user approves are safer than actions taken automatically.
How do we measure quality?	A set of real examples to test against, plus user feedback such as thumbs up and down.
What does it cost?	Cost per call × volume. Cache results, use smaller models where they're good enough.
How fast must it be?	Slow generation belongs in a queue, with the result shown when ready.
Whose data goes in?	Personal data, consent, retention, and what the model provider stores.

Say this

"I'd use AI where it saves people real time and where a mistake is cheap or easy to catch. For anything with numbers in, I'd have the system calculate them and let the AI only explain them."

19

PRODUCT INTERVIEW

Questions to ask David and Joseph

- How do you decide what gets built next? Where do ideas come from: clients, data, sales?
- How much do engineers shape the product, versus building to a spec?
- How do you know if a feature worked after it ships?
- Who are your main users day to day: clients' marketing teams, recruiters, your own account managers?
- What's the biggest product problem you're working on right now?
- How do product and engineering work together week to week?
- Where do you see AI taking the product over the next year?

Closing line for the product interview

"I liked working through that with you. I enjoy the part of engineering where you work out what's really needed before building it, and it sounds like that's how you work here."

20

BEFORE EACH INTERVIEW

Preparation checklist

Technical interview

1

Rehearse three stories out loud

Something you built (Typeset), a hard production problem, a technology you learnt fast.

2

Refresh Angular basics

Components, services, DI, RxJS and switchMap, signals. Enough to talk sensibly.

3 **Draw the AWS picture**

From memory: load balancer, ECS, RDS, ElastiCache, SQS, S3, CloudWatch.

4 **Practise the design question**

The weekly report email, out loud, in five minutes.

Product interview

5 **Use the product**

Look at their site and any demo. Who are the users? What would annoy them?

6 **Practise two scenarios**

Out loud: ask, reframe, options, first step, measure.

7 **Have paper ready**

Note their answers as you go; refer back to them.

Both days

8 **Check your setup**

Camera, sound, quiet room. No coding, but you may want to sketch.

9 **Know the names**

John and Matthew; then David and Joseph. Use them.

10 **Questions ready**

Three or four for each interview, from this guide.